Cancerous Tissue Classification Using the ARCENE Dataset

Project 4

1 Introduction

For the effective treatment of cancer, there is nothing more important than early diagnosis. However, many forms of cancer do not have straightforward diagnostic tests and diagnosis can thus be a lengthy process. The development of quick and effective tests that can be used to identify different types of cancer is crucial for accelerating diagnoses and improving the chances of treatment being effective.

A proposed rapid testing method is to obtain samples of potentially cancerous tissue from patients and use mass spectrometry to identify thousands of molecular features of a sample, from which it may be classified as cancerous or non-cancerous. In order to make this final prediction, a binary classification method has to be built from a large pool of tissue samples for which the classes (cancerous or non-cancerous) are known. ARCENE is a high-dimensional dataset, divided into training and test data for this purpose. For each observation, it provides the class of the sample along with $7,000$ features obtained through mass spectrometry and $3,000$ distractor features, known as probes. This paper aims to build a classification method using this dataset with which a tissue sample may be classified as cancerous, based on its biochemical features. The primary investigation of this paper, however, is whether it is possible for such a method to achieve an appropriate level of accuracy to make it a viable diagnostic test in the field of oncology.

2 Exploratory Data Analysis

The training data set has a high dimension and few observations. There are $n = 100$ observations each with $p = 2506$ variables, with no missing data. There are 44 cancerous samples and 56 non-cancerous samples, so neither class is over-represented. The training set contains a subset of variables from the test set, which has $q = 9920$ variables. The high dimension can make models computationally expensive, and having few observations can lead to overfitting. This motivates the need for dimension reduction, such as Principal Component Analysis (PCA) or multi-dimension scaling (MDS).

When performing dimension reduction techniques, the scale of the variables is important. Since PCA aims to find linear combinations of the original variables that retain as much variance as possible, if one variable has a uniquely large scale of variance it will be over-represented by PCA. The scale of variance in the training set varies greatly, as seen in Figure 1. Therefore, it is important to use the correlation matrix instead of the covariance matrix when performing PCA, and to scale the variables.

PCA can be susceptible to outliers. Due to the high dimension of the data it is difficult to define what an outlier would be here, so the whole training set is retained for PCA.

PCA works best on highly correlated data. The average correlation between variables is approximately 0.031. This is very low, but may be due to extremely small correlation between certain variables. It can be seen in Table 1 that there are many strongly correlated variables, indicating that PCA might be effective.

Correlation category	**Count**
Highly correlated	128,126
Mildly correlated	474,021
Lightly correlated	2,536,618

Table 1: Counts by correlation category (light < 0.33, mild < 0.67, high ≥ 0.67).

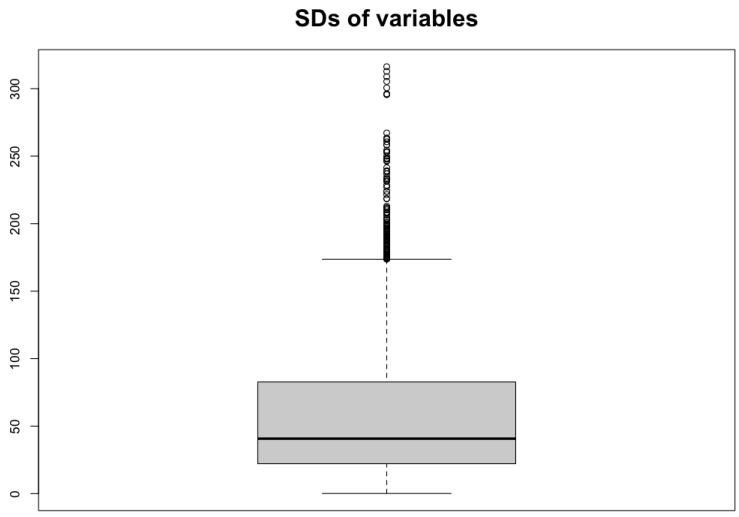

Figure 1: Box plot of standard deviation of variables in training set.

3 Methods

3.1 Dimension Reduction

To reduce the dimension of the training data one can perform PCA or MDS. For the purposes of classification, PCA is the chosen method as `predict()` can be used to map the test data onto the reduced dimensional space made up of the principal components (PCs) generated from the training data. There is no straightforward way of mapping the test set to the reduced space produced by performing MDS on the training set. The function `prcomp()` tends to be more stable than `princomp()` when there are more variables than observations. The parameter *scale* is set to `True` to ensure the variance is normalised. Performing PCA on the training set gives the following Scree plot, shown in Figure 2.

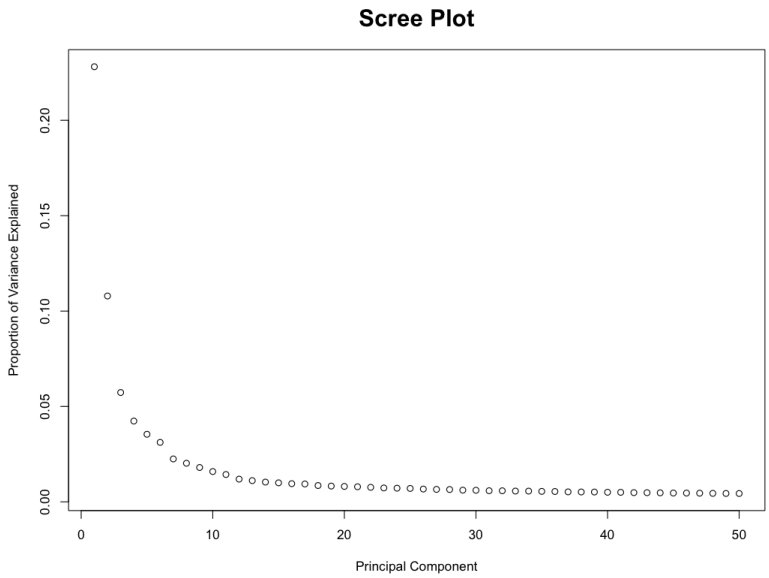

Figure 2: Scree plot for the first 50 principal components.

The elbow, where the curve starts to level out, appears to be around the 10th PC. However, this would only explain approximately 60% of the variability. Taking the first 40 PCs explains 80% of the variability, which is a good balance of reduced dimension and explanation power. This is the default number of PCs retained throughout the report.

3.2 Support Vector Machines

Support Vector Machines (SVM) are a supervised learning technique used to classify data. As with all supervised learning techniques, SVM learns the relationship between labelled inputs and outputs (where outputs are known for given inputs) and tries to predict the output of new data based on patterns previously detected.

Classifying the data involves fitting planar boundaries between groups, maximising the margin. The boundary line that separates the classes is a hyperplane, and the perpendicular distances from the hyperplane to the closest points in each class are the margins. Often, many hyperplanes may separate the classes; of these, the hyperplane chosen is the maximal margin hyperplane. While it is often most intuitive to picture SVM classification using a dataset with two classes and two variables, for which one can draw a straight line between groups, this is rarely the case in real data.

To extend the use of SVM beyond this basic case, the *Kernel Trick* is used when clean classes of data cannot be separated linearly in the variable space. The *Kernel Trick* involves transforming the data into a higher-dimensional space and finding a linear hyperplane that separates this transformed data. Upon transformation back to the original space, this hyperplane will appear non-linear.

In this study, three kernels were compared: a linear kernel, a radial basis function (RBF) kernel, and a polynomial of degree two. When the classes are not separable, SVM utilises soft margins, which are controlled in R via the cost parameter. The cost dictates the amount of misclassified points that are allowed within the model. A larger cost will result in a greater model penalty for misclassified data, creating a narrow margin; however, a high cost may incur overfitting on the data. Alternatively, a low cost parameter will allow more violations of the margin, leading to a more stable boundary.

The Arcene data set is particularly well-suited to SVM classification for two reasons. Firstly, there are more variables than observations which is suitable since the SVM parameters depend on the number of training observations, and not the number of features. Secondly, the response variable Class is binary. It takes the values -1 or 1 for non-cancerous or cancerous, respectively. This data structure aligns with the binary classification format that SVM is designed to tackle.

Before initiating model analysis, the training data were projected onto the first 40 principal components, which capture 80% of the variance in the training set (as discussed in Section 3.1). Using the principal components from PCA ensures that the variables are scaled and centred. The same PCA transformation was applied to the test set using the loadings derived from the training data to avoid data leakage.

The **svm()** function from the **e1071** package in R was used to fit all SVM models. To compare the different SVMs, the areas under their ROC curves (AUC) were used to calculate performance, as it measures how well the model predicts the correct output on the test set. Confusion matrices were also calculated, allowing for assessment of false negative rates, essential in this context due to the risk to health posed by the incorrect classification of a cancerous tumor.

To optimise based on kernel, all three kernels were tuned using 10-fold cross-validation via the `tune()` function in **e1071**, with cross-validation error rate used as the selection criterion at this stage. The linear kernel was tuned by comparing continuously narrowing cost ranges to identify the SVM with the lowest error value. The radial kernel takes both cost and gamma parameters, where gamma controls the localness of the kernel. Similarly to the linear framework, both were tuned in combination. For the polynomial kernel, cost, gamma and coef0 were tuned. See Table 2 for the tuned parameters and corresponding cross-validation error for each kernel.

Kernel	**Tuned Parameters**	**CV Error (10-fold)**	**AUC**
Baseline Linear	cost = 1 (Default Selection)	0.40	0.880
Tuned Linear	cost = 0.06	0.24	0.880
Radial (RBF)	cost = 1, gamma = 0.05	0.28	0.898
Polynomial (degree 2)	cost = 1, gamma = 0.05, coef0 = 2	0.22	0.886

Table 2: SVM Kernel Comparison: Tuned Parameters, Cross-Validation Error and corresponding AUC of ROC plot

Figure 3 shows the ROC curves for all three kernels, as well as the baseline SVM ROC curve. Visually, these curves are extremely similar, which is confirmed by all models having similar AUC

values as displayed in Table 2. This suggests that all models predict class to a comparable level of accuracy. In order to determine model is optimal, we may look to another metric, namely the false negative rate.

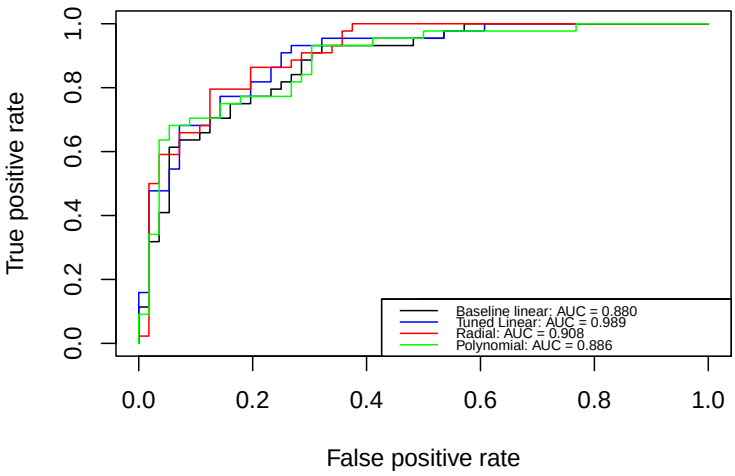

Figure 3: Overlayed ROC plot for comparison of ROC of different Kernels under consideration

Due to the similarity of each SVM's error and AUC, their confusion matrices are perhaps more telling in this study. As previously mentioned, a false negative corresponds to a cancerous tumor sample being classified as non-cancerous which may lead to a missed diagnosis, a potentially fatal mistake. In contrast, while false positives are still not ideal, they are less dangerous in this context; hence, it is the false negative rate which will be used to determine the optimal SVM approach in this study. The radial (RBF) kernel produces the fewest false negatives of the three models out of 100 cases, see Figure 4, making it the preferred SVM classifier for this problem, despite having the largest error when applied to the test set as seen in Table 3.

	Predicted	
Actual	**-1**	**1**
-1	50	6
1	15	29

(a) Baseline Linear

	Predicted	
Actual	**-1**	**1**
-1	48	8
1	10	34

(b) Tuned Linear

	Predicted	
Actual	**-1**	**1**
-1	37	19
1	3	31

(c) Radial (RBF)

	Predicted	
Actual	**-1**	**1**
-1	47	9
1	11	33

(d) Polynomial (deg. 2)

Figure 4: Confusion matrices for SVM models

Kernel	**Model Error**
Tuned Linear	0.18
Radial (RBF)	0.22
Polynomial (degree 2)	0.20

Table 3: SVM Kernel Comparison: Model Error When Determining Classes on Test Set

One limitation of non-linear SVM kernels, such as the RBF, is that they are non-parametric methods, meaning that uncertainty calculation for individual predictions is not straightforward. The radial kernel SVM is selected as the preferred model for this analysis since it produces the

fewest false negatives across all three kernels, making it the most appropriate choice in the context of cancer detection. Its test error of 0.22 and AUC of 0.898 are comparable to those of the linear and polynomial SVMs, confirming that this choice does not significantly affect the performance of the classification when compared to the other methods.

3.3 Classification Trees

Classification trees are a nonparametric classification method. They are very flexible, with no data assumptions or requirements, making them optimal for tuning to this dataset under different levels of dimension reduction. Classification trees inherently handle missing values, meaning that a tissue sample subject to a recording error with any one of the $7,000$ values obtained through mass spectrometry could still be classified with such a model. Crucially in this medical application however, classification trees are deemed to be the most easily interpretable binary classification method, as relatively simple diagrams can be produced illustrating how the final conclusion is reached. It is important that both doctors and patients can understand how a diagnosis has been reached.

Classification trees were grown for this data using both 40 and 50 PCs to perform dimension reduction, capturing 80% and 85% of variability respectively. A top-down approach was taken; growing a full tree with `rpart()`,= and then pruning it with `prune()`, using the minimum error criteria which minimises cross validation error. Minimum error was selected over the smallest tree criteria since minimising tree size is not necessarily beneficial with high dimensional data.

For both levels of dimension reduction a complexity parameter of 0.023 was selected during pruning, suggesting that their results would be similar. As we see in Figure 5 and Figure 6, these two trees are identical. When used to predict the class of test data, with corresponding PCA transformations applied, they both achieved an accuracy of 0.69. This means that whether we capture 80% or 85% of the variability in the data through dimension reduction makes no difference to the accuracy of this method, suggesting that there may be a cutoff point beyond which increasing the number of PCs has no benefit to the classification tree.

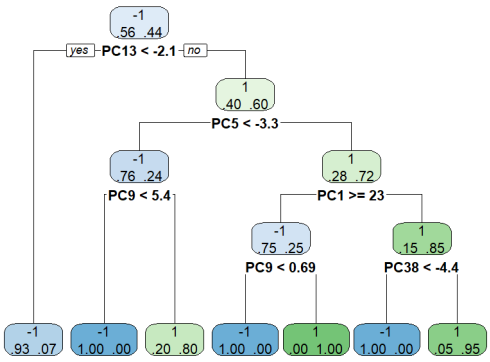

Figure 5: Tree with 40PC

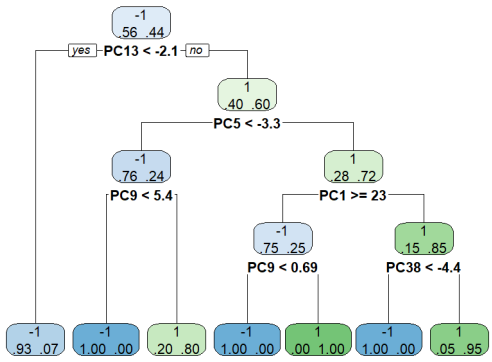

Figure 6: Tree with 50PC

In Figure 5 and Figure 6, we also observe that PC13 is used to make the split at both root nodes. By checking the variable importance of both trees using `variable.importance`, we see that PC13 is by far the most important variable within these trees (13.75), whilst few PCs above 13 hold signifcant importance.

To test whether removing unimportant variables improves the accuracy of a classification tree for this data, we repeat the above process using 13 PCs for dimension reduction. Whilst this choice of PCA only captures 62% of the variability of the data, it does agree with the Scree plot in Figure 2, which levels out at around 13.

This tree is pruned using a lower complexity parameter of 0.012, and in Figure 7, we see that this tree has a more complex structure than previous results. When used to predict the class of dimension-reduced test data, it achieves an accuracy of 0.70. While this is not a big improvement on previous trees, it suggests that capturing more variability through our dimension reduction is less important than using fewer, important variables when growing a classification tree for this data.

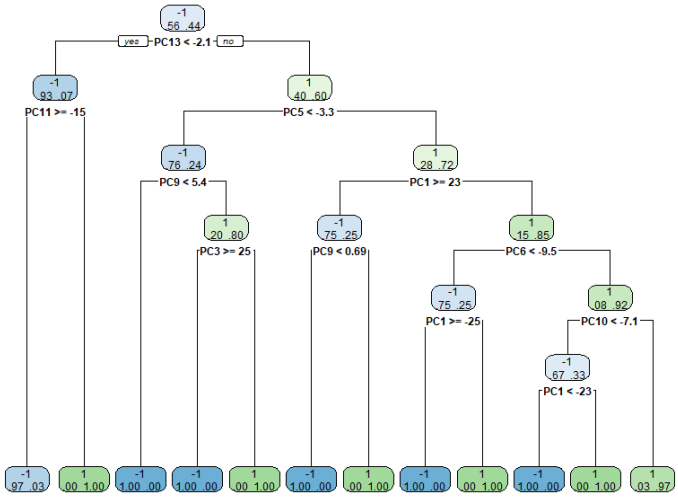

Figure 7: Tree with 13PC

3.3.1 Bagging

It is worth noting that while classification trees do have low bias, they are subject to overfitting and high variance. We turn to other tree based methods to combat this and see whether the predictive accuracy can be improved upon. There appears to be no benefit to including an extra 27 variables in a classification tree, so we proceed with 13 PCs.

Applying bagging (bootstrap aggregating) to a classification tree reduces variance by resampling observations to produce a large number of models, which are then averaged across into a single model. It is applied by using `randomForest()` and setting the argument *mtry* equal to the number of variables.

Applying bagging to the classification tree with 13 PCS achieves a test accuracy of 0.82, which is a significant improvement on our existing tree.

3.3.2 Random Forests

However, if we have observed anything so far it is that there appears to be a dominant feature in this data, which can lead to high correlation between bagged models, ultimately limiting the effect that bagging has on reducing variance. We will finally grow a random forest, which combats the effects of dominant variables by resampling observations and randomly excluding a subset of variables to produce a large number of models, then averaging across into a single model. This means that if there is a dominant feature, not all of the trees will use this feature to make their primary split, as is the case in bagging. Random forests are grown using `randomForest()` without specifying the argument *mtry*.

The random forest for the data with 13 PCs returns a test accuracy of 80%, which is a slight decrease from the bagged model.

The most effective way of visualising what happens inside these classification methods is by using `varImpPlot()` to plot variable importance, based on each feature's average reduction in the Gini index. Figure 8 shows the variable importance in the bagged model, where we see PC13 remains dominant. The variable importance within the random forest, as illustrated in Figure 9, demonstrates that the relative importance of all variables has been decreased, however PC13 is still significantly the most important.

Of the three tree based methods we have used, from their confusion matrices shown in Figure 10, we see that they all produce more false positives than false negatives which is optimal for this classification problem, with the bagged model and random forest model having false negative rates of 0.14 and 0.16 respectively. Not only do the bagged model and random forest model have higher accuracy than the classification tree, they both have lower variance making them preferable for the wider classification of tissue samples. However, the relatively small difference in their accuracy

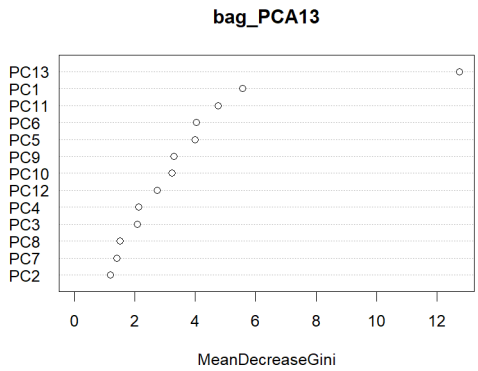

Figure 8: Bagged Model

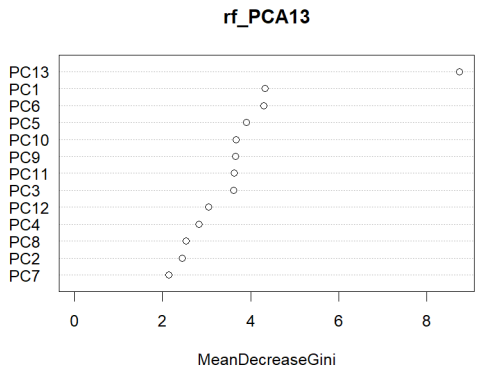

Figure 9: Random Forest Model

scores (80% vs 82%) means that selecting the optimal model should not be based on accuracy alone. The variable dominance in the bagged model raises concerns around how well this model would translate to other datasets. By using a random forest, minimal accuracy is compromised, however overfitting is even further reduced, suggesting that this may be the most appropriate tree based method for the practical classification of cancerous tissue.

	Predicted	
Actual	**-1**	**1**
-1	36	20
1	10	34

(a) Classification tree

	Predicted	
Actual	**-1**	**1**
-1	44	12
1	6	38

(b) Bagged model

	Predicted	
Actual	**-1**	**1**
-1	43	13
1	7	37

(c) Random forest model

Figure 10: Confusion matrices for tree based methods using 13 PCs

3.4 K-Nearest Neighbours

K-Nearest Neighbours(kNN) is a nonparametric classification method. For each point we look at its k-nearest labelled points, and predict the class of this point to be that of the majority of its neighbours. Once the optimal value of K is chosen, the distances between each test point and all the training points are calculated and sorted to determine the k-nearest neighbours. The class of the test point can then be predicted.

To apply kNN, it is standard to divide the data into training, validation, and test sets. However, in our case the dataset is extremely small: after applying PCA, we have only 100 observations and 40 PCs in the training set. If this were to be split into training and validation subsets, at least one subset would not contain enough observations to choose an optimal value of k and reliably estimate model performance. Using the standard 80/20 split, a validation set of only 20 points would lead to unstable accuracy estimates.

To address this problem, we use 10-fold cross-validation on the full training set. Cross-validation allows us to train and test the model multiple times on different subsets of the data, also providing a more stable estimate of accuracy using all available observations. Most importantly, it allows for the selection the optimal value of k (the number of neighbours) in a data-efficient way.

Once the best value of k is chosen using cross-validation, we then fit a final kNN model using this value in R, and evaluate its performance on the independent test dataset.

We applied 10-fold cross-validation to estimate the classification accuracy of KNN for K values from 1 to 100. The cross-validation plot shows that three values (K = 1, 8, and 13) achieved the same maximum CV accuracy, as shown in Figure 11 below.

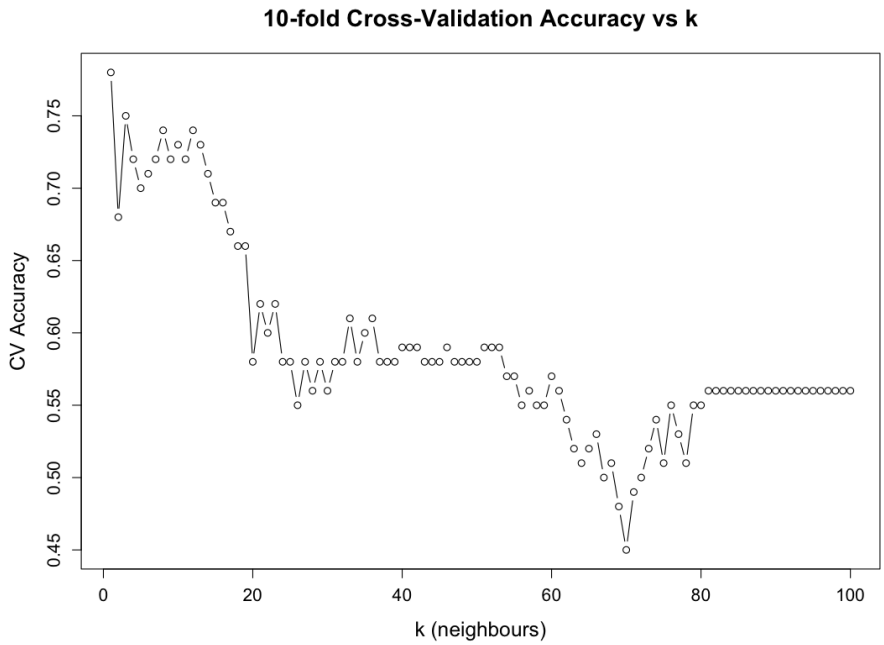

Figure 11: 10-fold Cross-Validation Accuracy for KNN

Since cross-validation alone could not distinguish between these tied values, we evaluated all three models on the independent test set. The test accuracies were 84% for K = 1, 74% for K = 8, and 72% for K = 13. Because K = 1 achieved the highest accuracy on unseen data, we selected K = 1 as the optimal number of neighbors.

This means each test point is classified using only its single nearest training point.

Metric	Value
Accuracy	0.84
Sensitivity	0.75
Specificity	0.82

Table 4: Metric table: KNN

	Predicted	
Actual	**-1**	**1**
-1	45	5
1	11	39

Table 5: Confusion matrix: KNN

We computed the confusion matrix, the sensitivity, specificity, and accuracy for the final KNN classifier using $K = 1$. The accuracy of 84% indicates that the KNN model correctly classifies most test observations. The sensitivity is 0.75, which means that the model correctly identifies 75% of cancerous samples (class 1). The specificity is 0.82, indicating strong performance in detecting non-cancer samples (class -1). Overall, the model performs slightly better at identifying healthy tissue than cancerous tissue, but sensitivity remains reasonably strong.

The confusion matrix summarises how the KNN classifier performs on the test set. For the non-cancerous class (-1), the model correctly predicts 45 samples and misclassifies 5 as cancerous. For the cancerous class (1), the model correctly identifies 39 samples and misclassifies 11 as non-cancerous.

3.5 Neural Networks

Neural networks (NNs) are a method for approximating any non-linear model. Inspired by neurons, they provide a framework for receiving data, running this data through layers of hidden nodes, and providing an output. In this case, a NN can be used to take as input the principal components of a sample of tissue, and return the probability that the sample is cancerous through the hidden nodes. Hidden nodes perform intermediate computations that allow the NN to learn complex patterns. Increasing the number of hidden nodes makes the model more flexible, but also increases the risk of overfitting.

The training data was split 80% training set and 20% validation set. The validation set was used to tune the number of principal components used and the number of hidden nodes. The best

combination is determined by validation accuracy, and ties are decided by choosing the simplest model to avoid overfitting. Due to the training set being reduced to 80 observations, the maximum number of principal components is 80. The Validation score, AIC, and BIC for each combination of 10 to 80 principal components and 5 to 15 hidden nodes was calculated. The best validation accuracy and simplest model is 17 PCs and 11 hidden nodes. The metrics are shown below in Table 6.

PCs	**Hidden**	**AIC**	**BIC**	**Validation Accuracy**
17	11	420	920	0.90

Table 6: Performance metrics of best model

This model has a test accuracy score of 87%. The confusion matrix is shown in Table 7.

	Predicted	
Actual	**-1**	**1**
-1	49	6
1	7	38

Table 7: Confusion matrix for NN on test set

The false positive rate and false negative rates are approximately equal, being 0.125 and 0.136 respectively. The sensitivity is 0.864 and specificity is 0.875. The model correctly identified 86.4% of positive samples and 87.5% of negative (non-cancerous) samples. The NN shows a balanced error rate, so it doesn't bias towards over or under classification. However, the presence of false negatives is cause for concern when diagnosing cancer.

3.6 Discriminant Analysis

3.6.1 Linear Discriminant Analysis

Linear Discriminant Analysis (LDA) is a classification technique that aims to separate classes by finding a linear boundary (straight line) that divides the groups. LDA assumes that each class follows a multivariate normal distribution with means specific to class, but shared covariance matrices which leads to the linear boundary. Since we have only two classes, LDA produces a single discriminant direction (LD1), which is the direction that best separates the groups.

Before fitting LDA, we check whether the classes have approximately equal covariance matrices. To do this, we compare the variance of each principal component across the two classes. If the within class variances are similar, the assumption of a shared covariance matrix is reasonable.

To assess the multivariate normality assumption required by LDA, we used `ggpairs()` to visualise the PCA components for each class. The diagonal panels show smooth density curves for each principal component, indicating that the marginal distributions are approximately normal. The off-diagonal scatterplots roughly show clouds of points for each pair of components, which is characteristic of data coming from a multivariate Gaussian distribution. Taken together, the diagonal and off-diagonal panels provide evidence that the multivariate normality assumption is satisfied.

After applying PCA , we first convert the PCs into a data frame and fit an LDA model with equal class prior probabilities of 1/2. The model then learns the linear combination of PCs that best separates cancerous and non-cancerous samples.

Metric	Value
Accuracy	0.79
Sensitivity	0.75
Specificity	0.82

Table 8: Metric table: LDA

	Predicted	
Actual	**-1**	**1**
-1	46	11
1	10	33

Table 9: Confusion matrix: LDA

After fitting the LDA model, it was used to predict the class of the test set. It achieved an accuracy of 84%, sensitivity of 0.75 and the specificity of 0.82.

The confusion matrix summarises how the LDA classifier performs on the test set. For the non-cancerous class (-1), the model correctly predicts 46 samples and misclassifies 11 as cancerous.

For the cancerous class (1), the model correctly identifies 33 samples but misclassifies 10 as non-cancerous.

We then visualise how well the samples separate along the first linear discriminant using density plots and LDA histograms (see Figure 12 and Figure 13). In both plots, the x-axis represents LD1 scores, which are the projections of each observation onto the linear discriminant direction. These show how well the classes separate along the discriminant axis and provide insight into the model's classification performance. Class 1 values are mostly negative, while class 1 values are mostly positive with a small overlap around LD1= 0. This indicates that LDA's linear boundary effectively separates the cancerous and non-cancerous groups.

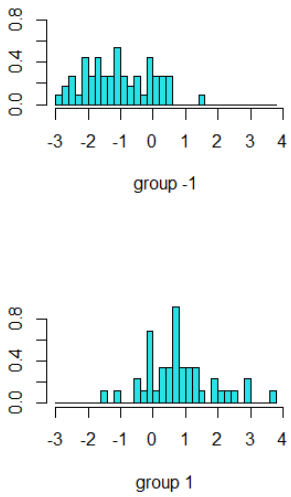

Figure 12: Class specific histograms of LD1 values

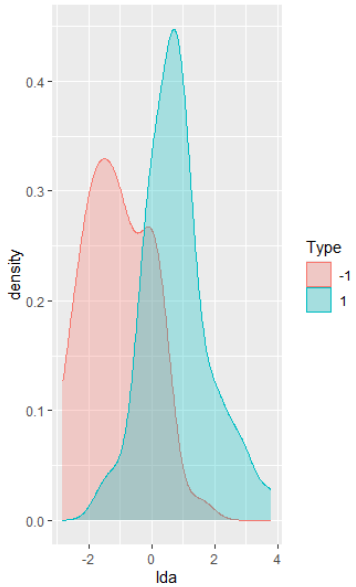

Figure 13: Density plot of LD1 scores for the two classes

3.6.2 Quadratic Discriminant Analysis

Although several PCs showed similar within-class variances, others displayed noticeable differences, indicating that the equal covariance assumption is only approximately satisfied. Therefore, while LDA is appropriate, we should also fit QDA.

Quadratic Discriminant Analysis (QDA) is similar to LDA, but it allows each class to have its own covariance matrix. This enables QDA to fit a curved decision boundary. After fitting QDA on the PCA transformed training data, we predicted the test set and calculated accuracy, sensitivity, specificity, and the confusion matrix. QDA is therefore evaluated solely through its classification results (accuracy, sensitivity, specificity).

Metric	Value
Accuracy	0.610
Sensitivity	0.886
Specificity	0.820

Table 10: Metric table: LDA

	Predicted	
Actual	**-1**	**1**
-1	55	38
1	1	6

Table 11: Confusion matrix: LDA

After fitting the LDA model, we predicted classes of the test set. We examine the metrics which result in the accuracy of 61%, the sensitivity is 0.88 and the specificity is 0.80

The confusion matrix summarises how the QDA classifier performs on the test set. For the non-cancerous class (-1), the model correctly predicts 55 samples and misclassifies 38 as cancerous. For the cancerous class (1), the model correctly identifies 6 samples but misclassifies 1 as non-cancerous.

4 Results and Conclusion

Table 12 displays a summary of the key test statistics calculated from the test set and used selection criteria for the classification method. The Neural Network method performs the best in the accuracy metric, obtaining 87% accuracy. The K-nearest-neighbour and Random Forest models also perform well on this metric, achieving 84% and 80%, respectively. The Radial RBF SVM also performs relatively well with 78% accuracy.

False negative rate is another key selection criterion. In the context of cancer diagnosis, a false negative corresponds to a cancerous tissue sample being classified as non-cancerous, representing a potentially life-threatening missed diagnosis. In this, the neural network again performs well with a false negative rate of 0.07. Compared to the other models, however, there are three other models (Radial SVM, Bagging, QDA), that perform better in this metric. With QDA specifically having only a 0.01 false negative rate, however, QDA has the lowest accuracy of all classification models in the study, and as such, we will not select this model. The second lowest False negative rate is the RBF SVM model.

The neural network achieves the highest overall accuracy, but it produces more false negatives than the Radial RBF SVM. The SVM, while slightly less accurate, produces less false negatives, which is a critical consideration in this diagnostic context. Furthermore, while neither NN or Radial RBF SVM are fully interpretable, SVM has a slight advantage over the NN model as in SVM, both the support vectors and boundary can be adjusted independently. However, in a neural network the prediction comes from many hidden weights interacting across layers. This makes the reasoning behind the decision difficult to understand.

Method	**Accuracy**	**Sensitivity**	**Specificity**	**False Negative Count**
SVM: Radial RBF	0.78	0.91	0.66	3
Classification tree	0.70	0.77	0.64	10
Bagging	0.82	0.86	0.79	6
Random forest	0.80	0.84	0.77	7
KNN ($k = 1$)	0.84	0.75	0.82	11
Neural network	0.87	0.86	0.88	7
LDA	0.79	0.75	0.82	10
QDA	0.61	0.86	0.59	1

Table 12: Summary of classification performance metrics for each method under consideration

In conclusion, the best model for identifying cancerous tissue is the support vector machine, using a radial basis function. While it does not achieve the highest overall accuracy, it provides the best balance between accuracy, sensitivity, and false negative rate, as shown in Table 12. Minimising false negatives is critical, and the SVM: Radial RBF achieves one of the lowest false negative counts while maintaining strong overall performance. For these reasons, the Radial RBF SVM is selected as the optimal model for this classification task.